

Tópicos Especiais em Inteligência Computacional Aplicada a Segurança de Sistemas

Aula 03

Programa de Pós-graduação em Computação Aplicada
Núcleo de Desenvolvimento Amazônico em Engenharia
Prof. Dr. Caio C. Moreira

Ciclo de um sistema baseado em ML aplicado à segurança

- Coleta de dados
 - Logs, tráfego de rede, arquivos executáveis, eventos de execução.
 - Bases públicas de malwares (amostras brutas).
 - Bases Públicas com Características Extraídas (datasets prontos para ML)
- Pré-processamento
 - Limpeza e normalização dos dados.
 - Conversão de logs/arquivos em representações numéricas.
 - Tratamento de classes desbalanceadas (SMOTE, undersampling).
- Extração de características (feature engineering)
 - Permissões solicitadas por aplicativos.
 - Chamadas de API/sistema.
 - Frequência de pacotes em tráfego de rede.
 - Strings e entropia em executáveis.
- Treinamento do modelo
 - Escolha do algoritmo (supervisionado ou não supervisionado).
 - Ajuste de hiperparâmetros.
- Avaliação do modelo
 - Divisão em treino/teste (ou validação cruzada).
 - Métricas: Acurácia, Precisão, Recall, F1-score, AUC-ROC.
- Interpretação de resultados para validar a utilidade prática.

Dataset de Teste

- Criação de um dataset sintético com python, pandas, numpy no Google Colab.

```
import pandas as pd
import numpy as np

data = {
    "id": range(1, 11),
    "nome": ["Alice", "Bob", "Carlos", "Diana", "Eva",
            "Felipe", "Gustavo", "Helena", "Iris", "João"],
    "idade": [25, np.nan, 32, 40, 29, 30, None, 45, 33, 36],
    "logins": ["2025-09-01 10:00", "2025-09-01 11:00",
              None, "2025-09-02 09:15", "2025-09-02 10:00",
              "2025-09-02 10:30", None, "2025-09-03 08:00",
              "2025-09-03 09:00", "2025-09-03 09:15"],
    "classe": [0, 0, 0, 1, 0, 0, 0, 1, 0, 0] }

df = pd.DataFrame(data)
df
```

Pré-processamento

- Limpeza e normalização dos dados
 - Dados reais costumam ter valores faltantes e escalas diferentes.
 - A limpeza corrige erros ou lacunas.
 - A normalização padroniza os valores, evitando que variáveis com escalas maiores dominem o modelo.

Pré-processamento

- Limpeza e normalização dos dados
 - Tarefas comuns
 - Remoção ou imputação de valores nulos

```
from sklearn.impute import SimpleImputer

# Imputação de valores ausentes (idade -> média)
imputer = SimpleImputer(strategy="mean")
df["idade"] = imputer.fit_transform(df[["idade"]])
df
```

Pré-processamento

- Limpeza e normalização dos dados
 - Tarefas comuns
 - Normalização de variáveis numéricas (ex.: Min-Max, Z-score)

```
from sklearn.preprocessing import MinMaxScaler

# Normalização de idade
scaler = MinMaxScaler()
df["idade_norm"] = scaler.fit_transform(df[["idade"]])
df
```

Pré-processamento

- Conversão de logs em representações numéricas
 -

```
# Conversão de logins para datetime
```

```
df["logins"] = pd.to_datetime(df["logins"])
```

```
# Extração de atributos temporais
```

```
df["dia"] = df["logins"].dt.day
```

```
df["mes"] = df["logins"].dt.month
```

```
df["ano"] = df["logins"].dt.year
```

```
# One-Hot Encoding de nomes
```

```
df_encoded = pd.get_dummies(df, columns=["nome"])
```

```
df_encoded
```

Pré-processamento

- Conversão de arquivos binários (.exe) em representações numéricas

```
# Caminho do arquivo binário (.exe)
file_path = "DOSPrinter_x64.exe"

# Função para converter em sequência de bytes (normalizada)
def file_to_byte_sequence(path, max_len=2000):
    with open(path, "rb") as f:
        byte_data = f.read()
    # Converte em inteiros (0-255)
    byte_array = np.frombuffer(byte_data, dtype=np.uint8)
    # Normaliza para [0,1]
    byte_array = byte_array / 255.0
    # Ajusta tamanho fixo (padding ou corte)
    if len(byte_array) > max_len:
        byte_array = byte_array[:max_len]
    else:
        byte_array = np.pad(byte_array, (0, max_len - len(byte_array)))
    return byte_array

seq_features = file_to_byte_sequence(file_path, max_len= 2000)

# Combinar em um DataFrame
df_exe_seq = pd.DataFrame([np.concatenate([seq_features])])
```


Pré-processamento

- Conversão de arquivos binários (.exe) em representações numéricas

```
# Caminho do arquivo binário (.exe)
file_path = "DOSPrinter_x64.exe"

# Função para extrair histograma de bytes
def file_to_byte_histogram(path):
    with open(path, "rb") as f:
        byte_data = f.read()
    byte_array = np.frombuffer(byte_data, dtype=np.uint8)
    # Histograma com 256 bins (um para cada valor de byte)
    hist, _ = np.histogram(byte_array, bins=256, range=(0, 256))
    # Normaliza pela contagem total
    hist = hist / hist.sum()
    return hist

hist_features = file_to_byte_histogram(file_path)
df_exe_hist = pd.DataFrame([np.concatenate([hist_features])])
```

Pré-processamento

- Conversão de arquivos binários (.exe) em representações numéricas

```
import matplotlib.pyplot as plt
import seaborn as sns

# df_exe_hist contém 1 linha e 256 colunas, cada coluna é um bin (valor de byte) e o valor é a frequência.
# Precisamos extrair esses valores para plotar o histograma.
byte_frequencies = df_exe_hist.iloc[ 0].values
byte_values = np.arange( 256)

plt.figure(figsize=( 15, 7))
sns.barplot(x=byte_values, y=byte_frequencies, color= 'skyblue')
plt.title('Distribuição de Frequência de Bytes do Arquivo .exe' )
plt.xlabel('Valor do Byte (0-255)' )
plt.ylabel('Frequência Normalizada' )
plt.xticks(np.arange( 0, 256, 16)) # Mostra rótulos a cada 16 bytes para melhor legibilidade
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Pré-processamento

- Tratamento de classes desbalanceadas
 - Problema comum: uma classe aparece muito mais que outra → viés no modelo.
 - Técnicas comuns:
 - SMOTE: gera novos exemplos sintéticos da classe minoritária.
 - Lembrar de remover valores nulos
 - Undersampling: reduz exemplos da classe majoritária.

```
from imblearn.over_sampling import SMOTE
```

```
X = df_encoded.drop(columns=["classe", "logins"])
```

```
y = df_encoded["classe"]
```

```
# SMOTE (oversampling)
```

```
X_res, y_res = SMOTE(random_state=42, k_neighbors=1).fit_resample(X, y)
```

Pré-processamento

- Tratamento de classes desbalanceadas
 - Problema comum: uma classe aparece muito mais que outra → viés no modelo.
 - Técnicas comuns:
 - SMOTE: gera novos exemplos sintéticos da classe minoritária.
 - Undersampling: reduz exemplos da classe majoritária.

```
from imblearn.under_sampling import RandomUnderSampler
```

```
#Undersampling
```

```
X_res2, y_res2 = RandomUnderSampler(random_state=42).fit_resample(X, y)
```

```
X_res2
```

Atividade 03

1. Escolher um dataset relacionado dataset de malwares de base pública e aplicar pelo menos as etapas de pré-processamento:
 - a. Limpeza de dados
 - b. Normalização ou padronização de atributos numéricos.
2. Caso utilize amostras brutas:
 - a. Conversão as amostras em representações numéricas.
 - b. Limpeza de dados
 - c. Normalização ou padronização de atributos numéricos.
3. Escrever um texto que poderá ser integrado no artigo final. O texto deve conter:
 - a. Descrição do dataset: origem, tamanho, principais variáveis.
 - b. Etapas de pré-processamento aplicadas.
 - c. O problema identificado nos dados (ex.: valores ausentes, classes desbalanceadas).
 - d. A técnica escolhida para resolver e sua fundamentação teórica (citação de referências).